

RAG from Scratch — System Reference

This document explains how this Retrieval-Augmented Generation system works internally: its retrieval pipeline, chunking strategy, storage, and safeguards against hallucination.

What Is Retrieval Augmented Generation

Retrieval Augmented Generation, or RAG, is a technique that combines a search step with a language-model generation step. Instead of relying only on what a language model learned during training, a RAG system first retrieves relevant passages from an external knowledge source, then gives those passages to the language model as context before it generates an answer. This keeps answers grounded in real, up-to-date, or private documents rather than the model's static training data, and reduces the model's tendency to invent information.

Parent-Child Chunking Strategy

This system splits documents into two chunk sizes rather than one. Small child chunks (roughly 500 characters, with 100 characters of overlap between consecutive chunks) are embedded and searched, because small chunks give precise, focused matches during vector similarity search. Each child chunk belongs to a larger parent chunk (roughly 2,000 to 10,000 characters), which is not embedded but is saved to disk as a JSON file. When a child chunk matches a query, the system loads its full parent chunk to give the language model more surrounding context than the small child chunk alone could provide. Searching small, generating from large is the core idea behind parent-child chunking.

Embedding Model Used For Vector Search

The system uses the sentence-transformers model all-MiniLM-L6-v2 to convert text into vectors. This model produces 384-dimensional embeddings and runs directly in-process on CPU or GPU, with no external API calls or HTTP requests required. Embeddings are normalized to unit length, so cosine similarity can be computed as a simple dot product. Documents are embedded in batches at indexing time for speed, and queries are embedded individually at search time.

How Conversation History Is Handled Across Turns

Conversation history is kept as a plain in-memory list of role and content pairs, one pair per user turn and one per assistant reply. The system keeps a rolling window of the most recent messages (10 messages by default) so prompts do not grow without bound. Before this window is trimmed, a summarizer compresses older turns into a one or two sentence summary covering the main topics discussed, key entities mentioned, and the most recent subject of conversation. This summary is passed to the query analysis step so that pronouns and vague references such as 'it' or 'that' can be resolved into concrete, self-contained questions before retrieval happens.

Handling Unclear Or Vague Queries

Before any retrieval happens, every incoming question is analyzed for clarity. Questions that are too vague to search for meaningfully, such as 'tell me more', 'what about it', or 'explain', are flagged as unclear. When a question is judged unclear, the system does not attempt retrieval at all. Instead, it

immediately returns a short, friendly clarification message asking the user what they meant, and waits for their next input rather than guessing or hallucinating an answer to an ambiguous question.

Splitting And Aggregating Sub-Questions

If a single user question actually contains multiple distinct topics — for example, asking about two unrelated things in one sentence — the query analysis step splits it into up to three separate, self-contained sub-questions. Each sub-question is then run through the full retrieval and answer pipeline independently, so each gets its own dedicated search and its own generated answer. If there was more than one sub-question, a final synthesis step combines all the individual answers into a single, coherent response, removing repeated information and connecting the pieces with natural language rather than presenting them as a numbered list of separate answers.

Similarity Threshold For Including A Chunk

During vector search, each candidate chunk receives a cosine similarity score between 0 and 1, where 1 means the chunk is identical in meaning to the query. The system only keeps chunks whose similarity score is at least 0.3, meaning at least thirty percent similar to the query. Chunks scoring below this threshold are discarded as not relevant enough to include in the context sent to the language model. By default the system retrieves up to seven child chunks per query before this threshold is applied.

How The System Avoids Hallucination

Several layers work together to keep answers grounded. First, the language model is given an explicit system instruction to answer using only the retrieved context and nothing from its own training knowledge, and to say plainly that it does not have enough information rather than guessing when the context does not cover the question. Second, after retrieval, a reflection step evaluates whether the retrieved context actually addresses the specific subject of the question, not just a loosely related topic; if it does not, the system retries with a different search tool or a rewritten query rather than generating from weak context. Third, after an answer is generated, a self-critique step checks whether the answer is actually grounded in the retrieved context and whether it strays into unsupported claims; if the critique finds problems, the answer is revised or replaced with an honest 'I don't have information about this' response before being shown to the user.

Database Used For Storing Vectors

All document vectors are stored in PostgreSQL using the pgvector extension, which adds a native vector column type and similarity search operators directly inside Postgres. An HNSW index is built on the embedding column so that nearest-neighbour search stays fast even as the number of stored chunks grows, instead of comparing the query against every row in the table. Using Postgres for vectors means the same database that stores everything else in the application can also serve as the vector store, with no separate vector database service required.

How To Add Your Own PDF Documents

To add new documents, copy the PDF files into the docs folder in the project directory. After adding files, trigger re-indexing, either by clicking the Re-index button in the web interface, uploading files directly through the interface's upload panel which triggers indexing automatically, or by running the `reindex` command from the interactive command-line tool. Re-indexing converts every PDF in the docs folder to markdown, splits the markdown into parent and child chunks, embeds the child chunks, and

stores everything fresh in Postgres, replacing whatever was indexed before.